

Занятие 0.1.

Занятие 0.1. Что мы строим

Head First Style

Файл: `methodological_guides/01_intro.md`

Спринт: 0

Время: 2 часа

Почему твой мозг сейчас сопротивляется

Ты слышал слова «рекомендательная система» и уже хочешь зевнуть. Мы это знаем. Поэтому **никакой теории** в первые 30 минут.

Вместо этого: ты **зайдёшь на сайт, потыкаешь его**, и только потом поймёшь, что это за штука. Как ребёнок — сначала трогает, потом спрашивает «а что это?».

 **Мозг работает так:** он не запоминает абстракции. Он запоминает **опыт**. Поэтому сначала — опыт, потом — слова.

План занятия

Мин	Что
0–15	Клонируем и запускаем
15–45	Тыкаем сайт. Сайт тыкает нас.
45–65	Что это за проект
65–85	Схема на доске
85–115	«Что должен делать алгоритм?»
115–120	Три волшебных слова

0–15 мин. «Клонируй это»

Диалог в зале

- Кто знает, что такое `git clone` ?
(лес рук)
- Отлично. Кто делал это сам, а не списывал у соседа?
(два пальца)
- Сегодня сделаем **все**. Прямо сейчас.

Преподаватель открывает терминал на проекторе. Студенты — у себя.

```
git clone https://github.com/AVick23/test_project.git
cd test_project
```

Осторожно! Если `git` не установлен — не паникуем. `sudo apt install git` (Linux) или <https://git-scm.com> (Windows/macOS). Преподаватель помогает.

Установка окружения

```
python3.12 -m venv .venv
source .venv/bin/activate          # Linux/Mac
# .venv\Scripts\activate           # Windows
pip install -r requirements.txt
```

Пока рір думает (2–3 мин):

- У программиста спрашивают: «Почему у тебя на столе 12 кружек?»
- «Потому что я не могу выбросить ни одну — вдруг там ещё есть кофе».
- Это, кстати, точная метафора кэша. Запомните её — сегодня пригодится.

Датасет

Скачиваем `ml-latest-small` с <https://grouplens.org/datasets/movielens/latest/>, распаковываем в `data/raw/ml-latest-small/`.

Нет интернета / забыл скачать / упал grouplens? Преподаватель раздаёт флешку. Не тормозим.

Заливка в базу

```
python scripts/seed.py --reset
```

30 секунд. В конце:

```
=====
SEED COMPLETE
=====
Movies:      9742
Users:       610
Interactions: 100836
=====
Password for seeded users: 'demo123'
=====
```

— 610 пользователей, пароль у всех `demo123`. Идеальная система безопасности. В реальной жизни так нельзя — но у нас учебная.

Финал

```
uvicorn backend.main:app --reload
```

Терминал замолкает.

— Готово. Сайт живой.

15–45 мин. «Потыкай это»

Каждый студент у себя в браузере.



Открываем

```
http://localhost:8000/app/
```

Тёмная тема. Стекланные карточки. 12 фильмов.

— Это вам не `print("Hello, world")`. Это **glassmorphism**, Apple-style. Потому что программисты тоже люди и хотят красиво.



Логинимся

Кнопка «Войти» → `user_1` / `demo123`.

В навбаре появляется `user_1`. Сайт как будто узнал.



Профиль

Переходим на `#/profile`.

Просмотрено: 272.

? Вопрос в зал:

Кто-нибудь из вас отметил 272 фильма?

(тишина)

Вот именно. Это **данные из MovieLens**. `user_1` уже посмотрел 272 фильма и поставил оценки. Мы просто залили это в базу.

Открываем фильм

`#/movie/1`

«Toy Story», 1995. Жанры, описание. Скроллим вниз — «Похожие фильмы».

? Вопрос:

В базе есть поле «похожие фильмы»?

Нет.

Так **кто** решил, что эти фильмы похожи?

(Пауза 3 секунды.)

Алгоритм. Тот самый, который вы будете писать весь семестр.

✓ Жмём «Смотрел»

Кликаем «✓ Смотрел» на любом фильме. Всплывает уведомление. Всё хорошо.

Возвращаемся на главную. Обновляем. Ничего не меняется.

🤖 Это момент истины.

Почему рекомендации не пересчитались, если я только что посмотрел фильм?

Преподаватель:

— Потому что `build_cache.py` не запускался.

Сайт не пересчитывает рекомендации при каждом клике — иначе он бы висел по 30 секунд.

Алгоритмы живут **отдельно**. Их запускают по расписанию — как уборщица в офисе, когда все ушли.

🧠 **Мозг запомнит:** «Алгоритмы — отдельно. Сайт — отдельно.»



Собираем реплики на доске

Вопрос в зал: «Что бросилось в глаза?»

- «Почему у всех одни и те же рекомендации?»
- «А где тут вообще алгоритм? Я его не вижу»
- «Почему после „Смотрел“ ничего не изменилось?»

Преподаватель:

— Отличные вопросы. **Все они будут в конце семестра.** Обещаю: вы сами на них ответите. Я пока не расскажу — вы должны помучиться.

45–65 мин. «Что это вообще за проект»

Задача, которую мы решаем

В базе **9742 фильма**. Человек за всю жизнь не посмотрит больше 2000.

Показать 9742 фильма — это то же самое, что **не показать ничего**.

Нужно выбрать **12 из 9742**.

Это и есть задача рекомендательной системы.

Три подхода

1. Popularity. «Что смотрят все».

Дёшево, работает всегда. Но показывает всем одно и то же. Ты любишь артхаус — а тебе показывают Форреста Гампа. Не беда, но скучно.

2. Content-based. «Что похоже на то, что тебе нравилось».

Смотрел «Матрицу» → покажем фильмы с похожими жанрами или описанием. Уже лучше. Но жанр — грубо. «Матрица» и «Интерстеллар» оба Sci-Fi, а фильмы разные.

3. Collaborative. «Что понравилось людям, похожим на тебя».

Если два человека высоко оценили 20 одних и тех же фильмов — они похожи. Что смотрит один, но не смотрел другой — рекомендуем второму. **Это то, чем Netflix зарабатывает миллиарды.**



Мозг запомнит: POPULARITY / CONTENT / COLLABORATIVE — три слова на весь курс.

Что мы будем делать

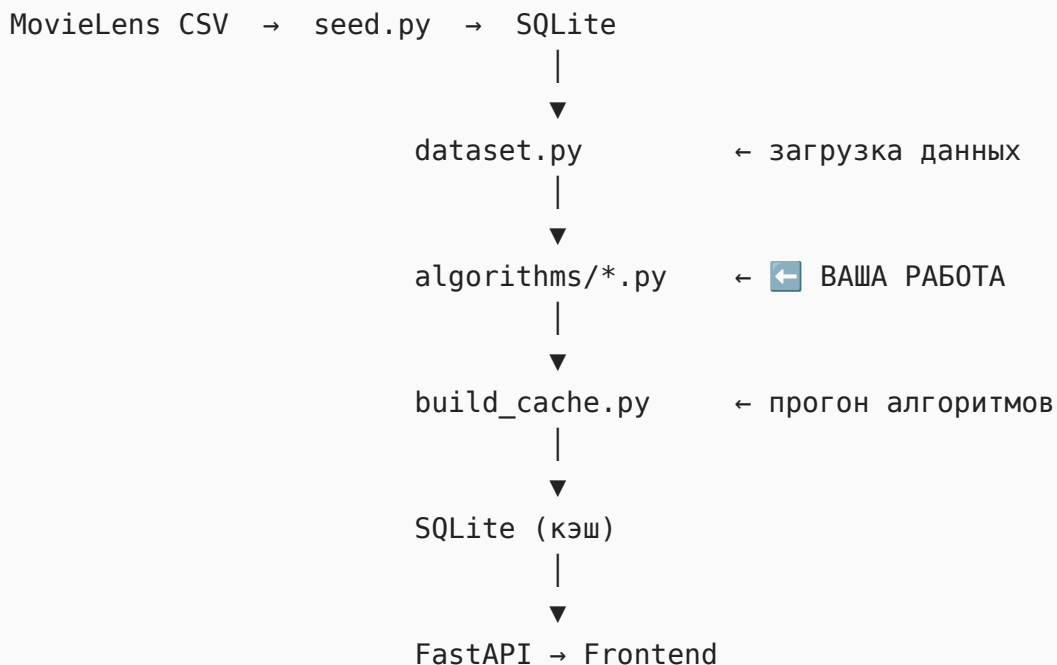
На доске — столбик:

```
Спринт 1 → registry.py      (каркас)
Спринт 2 → dataset.py       (данные)
Спринт 3 → popularity.py    (baseline)
Спринт 4 → content.py       (Jaccard + TF-IDF)
Спринт 5 → topk.py          (быстро достать топ-K)
Спринт 6 → metrics.py       (объективная оценка)
Спринт 7 → item_item.py     (collaborative)
Спринт 8 → mf.py            (matrix factorization)
```

Каждый спринт = один файл. Через 130 часов у вас **8 написанных модулей** и работающая рекомендательная система.

65–85 мин. Схема на доске

Преподаватель **рисует медленно**, комментирует вслух:



Три ключевые идеи

1. **«Готово»** — это всё **выше** и **ниже** `algorithms/`.
2. **«Ваша работа»** — модули внутри `algorithms/`.
3. **Алгоритм** — **чёрный ящик**. Получает `Dataset`, отдаёт список `(movie_id, score)`.

⚠ Алгоритм НЕ знает про SQL, FastAPI и браузер. И не должен.

? Контрольные вопросы

Q: Если вы поменяете `backend/services.py`, что произойдёт?

A: Сломается API. **Не трогаем.**

Q: А если поменяете `algorithms/jaccard.py`?

A: Поменяются рекомендации. **Это ваша работа.**

85–115 мин. «Что должен делать алгоритм?»

✎ Пиши ручкой в тетрадь. 5 минут.

Пишем на доске вопрос:

«Что должен делать алгоритм, чтобы вам было интересно им пользоваться?»

Собираем ответы без критики

На доске столбиком:

- «Показывать то, что понравится»
- «Не показывать уже просмотренное»
- «Быстро»
- «Разное для разных людей»
- «Не только хиты, но и что-то новое»
- «Учитывать мои оценки»
- «Если поставил 5 фильму про космос — показать ещё космос»

Раскладываем по трём колонкам

Popularity	Content-based	Collaborative
«быстро»	«похожее по жанрам»	«разное для разных»
«не смотрел ещё»	«учитывать оценки»	«что другим понравилось»
	«не только хиты»	«учитывать оценки»

Преподаватель:

— Мы сейчас **сами вывели** три подхода к рекомендациям.
Это не я вам сказал — это **вы написали**.
Так работает настоящая инженерия: сначала задача, потом метод.

115–120 мин. Итог

Три слова на доске крупно:

POPULARITY CONTENT COLLABORATIVE

— Запомните эти три слова. К концу курса вы напишете по ним **работающие алгоритмы**.
А сейчас — идите домой и потыкайте сайт ещё раз. Может, кто-то найдёт баг. Кто найдёт — тому печенюшка.



Домашнее задание

Никакого кода. Просто убедитесь, что у вас всё запускается.
Завтра — Git.



Что запомнит ТВОЙ МОЗГ из этого занятия

Понятие	Как запомнить
Recommendation system	Выбор 12 из 9742
Popularity	«Что смотрят все»
Content	«Похоже на то, что ты любишь»
Collaborative	«Что понравилось похожим людям»
Ваша работа	Только <code>algorithms/</code>
Алгоритм = чёрный ящик	Вход: Dataset. Выход: список фильмов.
Кэш	Ленивый студент: пока не ткнёшь — не пересчитает



Шпаргалка на случай аварии

Проблема	Что делать
git не установлен	<code>sudo apt install git</code> / https://git-scm.com
Python 3.11 нет	Поставить 3.12, всё совместимо
<code>pip install</code> висит	Раздать <code>requirements.txt</code> , пусть поставят дома
Нет датасета	Флешка + инструкция
<code>seed.py</code> падает на <code>tmdb_id</code>	<code>git pull</code> — уже исправлено
Порт 8000 занят	<code>--port 8001</code>
Браузер показывает старое	Ctrl+Shift+R

Если кто-то сильно опережает:

— Уже запустил? Открой `/docs` — там Swagger UI. Потыкай эндпоинты. Потом расскажешь, что нашёл.